Placement.OS - Viva Defense Q&A

Q1: Why did we deploy the Frontend on Netlify rather than Vercel?

Because the frontend is built as a Single Page Application using React and Vite, it doesn't require complex Server-Side Rendering (SSR). Vercel is highly opinionated toward Next.js workloads. Netlify is framework-agnostic, provided seamless Vite integration, automatic CI/CD, and an aggressive Global Edge CDN to deliver the compiled frontend decoupled from the ML backend without unnecessary overhead.

Q2: How does the application calculate the Probability Score versus the Readiness Score?

Probability Score: This is an overall statistical prediction calculated via a Heuristic Weighted Algorithmic Model. We assigned exact mathematical weights to profile metrics (e.g., CGPA = 0.5, Internships = 1.2). The raw sum is passed through a Sigmoid Activation Function to graph the score onto a normalized 0-to-100 percentage curve.

Readiness Score: This evaluates how ready you are right now for your specific Target Designation. It is calculated by taking a base score and subtracting a rigid penalty for every single prerequisite Node you are structurally missing within our Directed Acyclic Knowledge Graph.

Q3: How does the application predict target companies?

The company recommendation engine operates as a rule-based AI filtering pipeline. It first drops any companies where the user's CGPA strictly fails the company's minimum threshold. Then, it uses a Skill Synergy Engine to intersection-check the user's acquired skills against the company's array of required skills, calculating a raw match percentage. It aggregates bonuses for high CGPA and experience, dropping any company with less than a 25% match synergy, and returns the top 6 highest-scoring targets.

Q4: How does it calculate the Skill Gap?

The Skill Gap utilizes a Neuro-Symbolic AI approach spanning two layers:

1. Neural (Semantic Vector Embeddings): Using Hugging Face's sentence-transformers, the user's raw text inputs are translated into mathematical vectors and mapped onto canonical skills in our database via Cosine Similarity.

2. Symbolic (Directed Acyclic Graph): A Knowledge Graph was built using NetworkX. A Shortest Path Algorithm traces the route from the skills the user already possesses to the ultimate node of their Target Designation. The traversed nodes in between represent the precise 'Skill Gap'.

Q5: How does the application generate the 12-week roadmap, and how reliable is it?

The backend takes the exact list of missing nodes (the Skill Gap) outputted by the Shortest Path Algorithm

and mathematically divides the 12 weeks among them. Because it relies on a deterministic Directed Acyclic Graph (DAG) rather than a hallucinating Generative LLM, it is highly reliable structurally. The graph strictly enforces prerequisite logic, ensuring the generated syllabus is mathematically sequential rather than randomly generated.

Q6: Why is the Frontend built on React and Vite?

Why React? Building a complex Dashboard requires heavy state management. Using pure HTML/JS would result in a messy codebase. React provides a strict Component-Based Architecture and entirely relies on the Virtual DOM, instantly updating complex UI pieces mathematically without ever reloading the browser page.

Why Vite? While traditional Webpack tools crawl and bundle the entire application slowly, Vite uses 'esbuild' (written natively in Go) to pre-bundle dependencies up to 100x faster utilizing Native ES Modules. We chose Vite exclusively for compilation velocity, instantaneous local hot-module reloading, and to output an incredibly lightweight minified production package.

Q7: What is the exact data flow architecture? What happens when a user inputs data till output limit?

1. Frontend Input: The user's typed data is stored in React's Virtual State. On compile, it is packaged into a JSON Object and sent securely over the network via Axios.

2. Backend Ingestion: The Flask API catches the JSON payload, validates it, and uses the SQLAlchemy ORM to save it persistently into the SQLite database.

3. AI Pipeline Processing: Once on the Dashboard, the Flask Orchestrator pulls the user profile and fires the data simultaneously into three mathematical pipelines: The Probability Sigmoid Model, the Company Filtering Engine, and the Neuro-Symbolic Vector Engine.

4. Render Payload: The backend bundles the outputs of all three pipelines into a single heavy JSON payload. React intercepts the data, injects it into Tailwind CSS models, renders radar charts via Recharts, and instantly visualizes the complete interface.

Q8: Where did you retrieve the dataset from?

We intentionally engineered the system to NOT rely on a traditional, static tabular dataset (like a Kaggle CSV file) because tech industry requirements evolve far too rapidly and tabular data decays quickly.

Instead, we used:

1. Pre-Trained Foundational Models: We leveraged Hugging Face's foundational NLP model, natively

pre-trained on over 1-billion data points, giving it an out-of-the-box mathematical understanding of tech terminology without needing retraining.

2. Proprietary Knowledge Graph: We manually aggregated current, real-world data directly from modern job descriptions across top-tier tech firms to extract strict CGPA cutoffs and tech-stacks. We directly codified these real-time metrics structurally into our NetworkX Directed Acyclic Graph.